Project Phase 2 — Statistical Damage Detection

Course: CIV E 789 Sensing Techniques and Data Analytics (Fall 2026)
Instructor: Mohammad Talebi-Kalaleh, Ph.D.
Group: 6 (sensors S2, S4, S6, S9, S12, S15, S18, S21, S24)
Data: `phase2_modal_parameters.json` (intact, damage1, damage2; 20 estimates each)

Contents

0. Setup
1. Part A: Feature Distribution Analysis
2. Part B: Single Feature Damage Detection
3. Part C: Bivariate Analysis
4. Part D: Feature Fusion and Engineering
5. Part E: Principal Component Analysis
6. Part F: Engineering Insights
7. Use of Generative AI

0. Setup

This section loads the data used throughout the project and sets up the common tools. Parts A–E start from the arrays created here, so that all of them work with exactly the same data.

- **0.1** Libraries, paths and helper functions to present tables and figures.
- **0.2** Loading the JSON file and the group 6 sensors.
- **0.3** Extraction of natural frequencies and mode shapes, with automatic checks.

0.1 Libraries, paths and presentation helpers

- The file `phase2_modal_parameters.json` is searched for first in the notebook folder and, if it is not there, in the parent folder.
- Figures are saved in `figures/part-a/` … `figures/part-e/` and result tables in `results/` (CSV). The folders are created if they do not exist.
- `show_table` places the table caption above the table (**Table N.**) and `show_figure` places the figure caption below the figure (**Figure N.**), both outside the object. `show_figure` also saves the figure in the folder of its part.

```
import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from IPython.display import display, Markdown

# Input data: next to this notebook or, if not there, in the parent folder
```

```
DATA_FILE = 'phase2_modal_parameters.json'
DATA_PATH = next((p for p in (Path(DATA_FILE), Path('..') / DATA_FILE) if p.exists()),
None)
assert DATA_PATH is not None, f'{DATA_FILE} not found next to the notebook or in its
parent folder'

# Output folders: one figure folder per part, one folder for result tables
FIG_DIR = Path('figures')
RES_DIR = Path('results')
for part in ['part-a', 'part-b', 'part-c', 'part-d', 'part-e']:
    (FIG_DIR / part).mkdir(parents=True, exist_ok=True)
RES_DIR.mkdir(exist_ok=True)


def show_table(df, number, caption, filename=None, precision=4):
    """Show the table caption above the table; optionally save the table as CSV in
results/."""
    display(Markdown(f'**Table {number}.** {caption}'))
    display(df.style.format(precision=precision))
    if filename is not None:
        df.to_csv(RES_DIR / filename)


def show_figure(fig, number, caption, part, filename):
    """Save the figure in figures/<part>/, show it and put the caption below it."""
    fig.savefig(FIG_DIR / part / filename, dpi=300, bbox_inches='tight')
    display(fig)
    plt.close(fig)
    display(Markdown(f'**Figure {number}.** {caption}'))
```

0.2 Data loading and group 6 sensors

- The JSON file has two blocks: `scenarios` (data for `intact`, `damage1` and `damage2`) and `metadata` (sampling information, sensors assigned to each group and position of each sensor).
- The group sensors are read from `metadata.sensor_assignments.group_6` instead of being typed by hand, to avoid copying errors.
- Each mode shape is a list of 24 values ordered from S1 to S24, so sensor S_k is at index $k-1$ of the list (Table 1).
- The position of each sensor (`metadata.sensor_positions_m`) will be used as the x-axis of the mode shapes.

```
with open(DATA_PATH, 'r') as f:
    data = json.load(f)

GROUP = 6
SCENARIOS = ['intact', 'damage1', 'damage2']
MODES = ['mode_1', 'mode_2', 'mode_3']
N_MEAS = data['metadata']['sampling_info']['total_estimates']   # 20 estimates per
condition
N_SENSORS_TOTAL = len(data['metadata']['sensor_positions_m'])   # 24 sensors on the
bridge

# Sensors assigned to our group, read from the file (not typed by hand)
```

```
my_sensors = data['metadata']['sensor_assignments'][f'group_{GROUP}']
SENSOR_IDX = np.array([int(s[1:]) - 1 for s in my_sensors])   # Sk -> list index k - 1
SENSOR_POS = np.array([data['metadata']['sensor_positions_m'][s] for s in my_sensors])
# [m]

sensors_table = pd.DataFrame({'List index': SENSOR_IDX, 'Position (m)': SENSOR_POS},
                             index=pd.Index(my_sensors, name='Sensor'))
show_table(sensors_table, 1, f'Sensors assigned to group {GROUP}, their index in the
24-value lists '
           'and their position along the bridge.', precision=1)
```

Table 1. Sensors assigned to group 6, their index in the 24-value lists and their position along the bridge.

Sensor	List index	Position (m)
S2	1	7.5
S4	3	17.5
S6	5	27.5
S9	8	42.5
S12	11	57.5
S15	14	72.5
S18	17	87.5
S21	20	102.5
S24	23	117.5

0.3 Extraction of natural frequencies and mode shapes

The first cell defines the two extraction functions from the assignment starter code (Section 7), with the same names and behaviour: `extract_frequencies(scenario, mode_key)` returns the 20 frequencies of one mode, and `extract_mode_shape(scenario, measurement_idx, mode_key)` returns one mode shape at the 9 group sensors. The second cell uses them to store the data in numpy arrays, one per scenario (Table 2).

Table 2. Numpy arrays created in the data extraction.

Array	Size	Content
`freqs[scenario]`	20 × 3	measurements × modes, natural frequency in Hz
`shapes[scenario]`	3 × 20 × 9	modes × measurements × group 6 sensors

- **Assumption:** value i of each frequency list and `measurement_i` of the mode shapes come from the same 30-s segment. The assignment does not state this explicitly. It does not matter for Parts A and B, which analyse each feature separately, but Parts C, D and E combine several features of the same measurement into a single point.

- Measurements are read as `measurement_1` … `measurement_20` in numeric order, not in the order of the JSON keys.
- Mode shapes are not normalized again: they are already scaled to a maximum of 1 over the 24 sensors. The MAC does not depend on scale, so this does not affect it.
- The checks (`assert`) stop the notebook if a measurement is missing, if a mode shape does not have 24 values, or if there are missing values (NaN).

```
# Extraction functions from the starter code (same names and behaviour)

# Extract frequencies for a scenario
def extract_frequencies(scenario, mode_key='mode_1'):
    return np.array(data['scenarios'][scenario]['natural_frequencies'][mode_key])


# Extract mode shape at assigned sensors
def extract_mode_shape(scenario, measurement_idx, mode_key='mode_1'):
    mode_data = data['scenarios'][scenario]['mode_shapes'][mode_key]
[f'measurement_{measurement_idx}']
    # mode_data is a list of 24 values ordered S1..S24, so S1 -> index 0
    return np.array([mode_data[int(s[1:]) - 1] for s in my_sensors])
```

```
# freqs[scenario]:  (N_MEAS, 3) array -> measurements x modes, in Hz
# shapes[scenario]: (3, N_MEAS, 9) array -> modes x measurements x our 9 sensors
# Measurements are read as measurement_1 ... measurement_20 (numeric order), so row i
of
# freqs and measurement i of shapes refer to the same 30-s segment.
freqs = {}
shapes = {}
for sc in SCENARIOS:
    freqs[sc] = np.column_stack([extract_frequencies(sc, m) for m in MODES])
    shapes[sc] = np.array([[extract_mode_shape(sc, i, m) for i in range(1, N_MEAS +
1)]
                           for m in MODES])

# Sanity checks on the raw file and on the extracted arrays
for sc in SCENARIOS:
    for m in MODES:
        assert len(data['scenarios'][sc]['natural_frequencies'][m]) == N_MEAS
        mode_shapes = data['scenarios'][sc]['mode_shapes'][m]
        assert set(mode_shapes) == {f'measurement_{i}' for i in range(1, N_MEAS + 1)}
        assert all(len(v) == N_SENSORS_TOTAL for v in mode_shapes.values())
    assert freqs[sc].shape == (N_MEAS, len(MODES))
    assert shapes[sc].shape == (len(MODES), N_MEAS, len(my_sensors))
    assert np.isfinite(freqs[sc]).all() and np.isfinite(shapes[sc]).all()
print('Extraction checks passed: '
      f'{len(SCENARIOS)} scenarios x {len(MODES)} modes x {N_MEAS} measurements x
{len(my_sensors)} sensors')
```

```
Extraction checks passed: 3 scenarios x 3 modes x 20 measurements x 9 sensors
```

```
# One table per scenario: rows = modes, columns = measurements
for t, sc in enumerate(SCENARIOS, start=3):
    freq_table = pd.DataFrame(freqs[sc].T,
                              index=pd.Index([f'f{k + 1} (Hz)' for k in
range(len(MODES))], name='Mode'),
```

```
                                columns=pd.RangeIndex(1, N_MEAS + 1,
name='Measurement'))
    show_table(freq_table, t, f'Natural frequencies of modes 1–3 for each of the 20
measurements, {sc} scenario.',
               precision=3)
```

Table 3. Natural frequencies of modes 1–3 for each of the 20 measurements, intact scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.063	1.274	1.259	0.973	1.154	1.276	…	1.091	1.095	1.091	1.327	1.402
f2 (Hz)	2.847	3.404	3.253	2.537	3.052	3.345	…	2.875	2.801	2.855	3.450	3.733
f3 (Hz)	4.083	4.945	4.795	3.677	4.330	4.912	…	4.085	4.164	4.126	4.961	5.368

Table 4. Natural frequencies of modes 1–3 for each of the 20 measurements, damage1 scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.131	1.191	1.262	1.052	0.994	1.232	…	1.098	1.154	1.082	0.988	0.966
f2 (Hz)	2.777	2.962	3.067	2.769	2.429	3.077	…	2.778	2.852	2.790	2.549	2.569
f3 (Hz)	4.284	4.506	4.717	4.059	3.738	4.581	…	4.026	4.319	4.054	3.708	3.906

Table 5. Natural frequencies of modes 1–3 for each of the 20 measurements, damage2 scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.212	1.052	1.095	1.054	1.064	1.035	…	0.981	1.159	1.096	0.917	1.255
f2 (Hz)	2.793	2.557	2.841	2.533	2.621	2.488	…	2.555	2.955	2.681	2.338	3.016
f3 (Hz)	4.491	4.064	4.379	3.992	3.980	4.107	…	3.802	4.416	4.161	3.572	4.662

Part A: Feature Distribution Analysis (25 pts)

A1. Natural Frequency Distributions (8 pts)

A2. Mode Shape Visualization (9 pts)

A3. Modal Assurance Criterion (MAC) Distributions (8 pts)

Part B: Single Feature Damage Detection (20 pts)

B1. Z-Score Anomaly Detection (10 pts)

B2. Wasserstein Distance Analysis (7 pts)

B3. Feature Sensitivity Ranking (3 pts)

Part C: Bivariate Analysis (20 pts)

C1. Joint Distribution Visualization (7 pts)

Se analizan dos pares de variables: (f_1, f_2) y $(\mathrm{MAC}_1, \mathrm{MAC}_2)$.

Para calcular el Modal Assurance Criterion (MAC) se utilizó la siguiente ecuación:

$$\mathrm{MAC} = \frac{\left(\boldsymbol{\phi}_{ref}^{T}\,\boldsymbol{\phi}_{test}\right)^{2}}{\left(\boldsymbol{\phi}_{ref}^{T}\boldsymbol{\phi}_{ref}\right)\left(\boldsymbol{\phi}_{test}^{T}\boldsymbol{\phi}_{test}\right)} \tag{1}$$

Donde:

- $\boldsymbol{\phi}_{ref}$: forma modal de referencia, igual al promedio de las 20 mediciones del escenario intacto (9 valores, uno por sensor).
- $\boldsymbol{\phi}_{test}$: forma modal de una medición (9 valores).
- $\boldsymbol{\phi}^{T}\boldsymbol{\phi}$: producto punto, es decir, se multiplica sensor por sensor y se suman los resultados.

```
# MAC (Eq. 1) of each measured mode shape against the intact reference, for modes 1
and 2
def calculate_MAC(phi_ref, phi_test):
    numerator = np.abs(np.dot(phi_ref, phi_test))**2
    denominator = np.dot(phi_ref, phi_ref) * np.dot(phi_test, phi_test)
    return numerator / denominator


MAC_MODES = [0, 1]                                  # mode 1 and mode 2 (indices in
MODES)
ref_shapes = shapes['intact'].mean(axis=1)          # (3, 9): mean of the 20 intact
shapes of each mode

# mac[scenario]: (N_MEAS, 2) array -> measurements x [MAC1, MAC2]
mac = {sc: np.array([[calculate_MAC(ref_shapes[j], shapes[sc][j, i]) for j in
MAC_MODES]
                     for i in range(N_MEAS)])
       for sc in SCENARIOS}

# One table per scenario: rows = MAC of each mode, columns = measurements
for t, sc in enumerate(SCENARIOS, start=6):
    mac_table = pd.DataFrame(mac[sc].T,
                             index=pd.Index([f'MAC{j + 1}' for j in MAC_MODES],
name='Mode'),
```

```
                              columns=pd.RangeIndex(1, N_MEAS + 1, name='Measurement'))
    show_table(mac_table, t, f'MAC of modes 1 and 2 for each of the 20 measurements,
{sc} scenario '
               '(reference: mean intact mode shape).', precision=4)


# Scatter of (f1, f2) for each scenario; same axis limits in all panels to compare
them
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharex=True, sharey=True)
for ax, sc, label in zip(axes, SCENARIOS, ['(a)', '(b)', '(c)']):
    ax.scatter(freqs[sc][:, 0], freqs[sc][:, 1], s=30, alpha=0.8)
    ax.set_title(f'{label} {sc}')
    ax.set_xlabel('$f_1$ (Hz)')
    ax.grid(alpha=0.3)
axes[0].set_ylabel('$f_2$ (Hz)')
fig.tight_layout()
show_figure(fig, 1, 'Natural frequencies of mode 1 ($f_1$) versus mode 2 ($f_2$) for
the 20 measurements of each '
            'scenario: (a) intact, (b) damage1, (c) damage2.', 'part-c',
'C1_f1_f2_scatter.png')

# Scatter of (MAC1, MAC2) for each scenario; each panel scaled to its own data
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, sc, label in zip(axes, SCENARIOS, ['(a)', '(b)', '(c)']):
    ax.scatter(mac[sc][:, 0], mac[sc][:, 1], s=30, alpha=0.8)
    ax.set_title(f'{label} {sc}')
    ax.set_xlabel(r'$\mathrm{MAC}_1$')
    ax.set_ylabel(r'$\mathrm{MAC}_2$')
    ax.xaxis.set_major_locator(plt.MaxNLocator(5))   # fewer ticks so the labels do
not overlap
    ax.grid(alpha=0.3)
fig.tight_layout()
show_figure(fig, 2, r'MAC of mode 1 ($\mathrm{MAC}_1$) versus MAC of mode 2
($\mathrm{MAC}_2$) for the 20 measurements '
            'of each scenario: (a) intact, (b) damage1, (c) damage2.', 'part-c',
'C1_MAC1_MAC2_scatter.png')
```

Table 6. MAC of modes 1 and 2 for each of the 20 measurements, intact scenario (reference: mean intact mode shape).

Measurement	1	2	3	4	5	…	17	18	19	20
Mode										
MAC1	1.0000	0.9999	0.9999	0.9999	0.9999	…	0.9992	1.0000	0.9999	0.9999
MAC2	0.9994	0.9991	0.9994	0.9997	0.9952	…	0.9985	0.9991	0.9980	0.9994

Table 7. MAC of modes 1 and 2 for each of the 20 measurements, damage1 scenario (reference: mean intact mode shape).

Measurement	1	2	3	4	5	…	17	18	19	20
Mode										
MAC1	0.9996	0.9994	0.9990	0.9995	0.9992	…	0.9992	0.9999	0.9996	0.9997
MAC2	0.9905	0.9932	0.9854	0.9982	0.9883	…	0.9925	0.9880	0.9954	0.9906

Table 8. MAC of modes 1 and 2 for each of the 20 measurements, damage2 scenario (reference: mean intact mode shape).

Measurement	1	2	3	4	5	…	17	18	19	20
Mode										
MAC1	0.9971	0.9983	0.9987	0.9966	0.9992	…	0.9995	0.9981	0.9982	0.9992
MAC2	0.9719	0.9748	0.9890	0.9738	0.9895	…	0.9963	0.9758	0.9854	0.9789

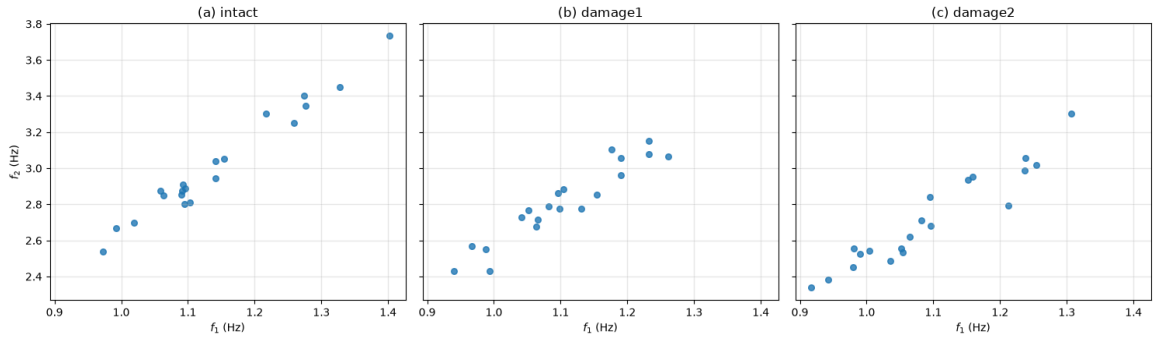


Figure 1. Natural frequencies of mode 1 (f_1) versus mode 2 (f_2) for the 20 measurements of each scenario: (a) intact, (b) damage1, (c) damage2.

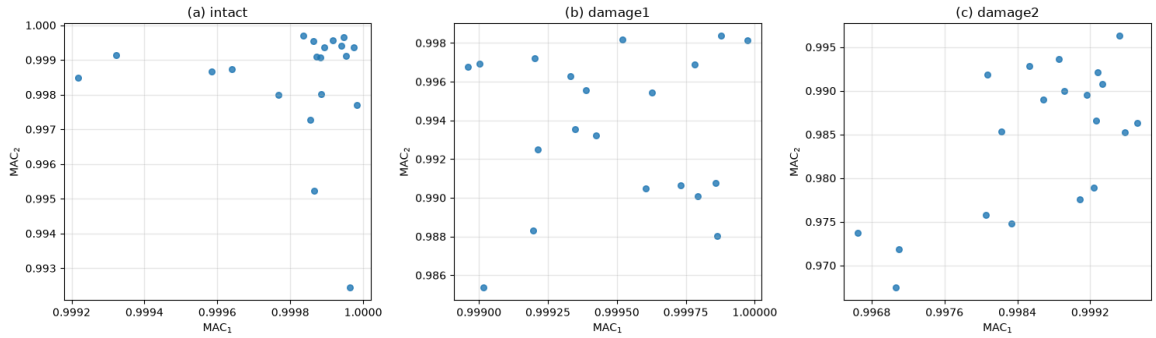


Figure 2. MAC of mode 1 (MAC_1) versus MAC of mode 2 (MAC_2) for the 20 measurements of each scenario: (a) intact, (b) damage1, (c) damage2.

El contorno de la gaussiana de dos variables se calcula a través de la siguiente formula:

$$(\mathbf{x} - \boldsymbol{\mu}_0)^T \, \boldsymbol{\Sigma}_0^{-1} \, (\mathbf{x} - \boldsymbol{\mu}_0) = \chi^2_{p,\,1-\alpha}$$

Donde:

- $\mathbf{x}$ es el conjunto de puntos de la linea base.
- $\boldsymbol{\mu}_0$ es el vector de medias de la línea base.
- $\boldsymbol{\Sigma}_0^{-1}$: la inversa de la matriz de covarianza de la línea base.
- $\chi^2_{p,\,1-\alpha}$: el valor de la chi-cuadrado con p grados de libertad, uno por característica, al nivel de confianza $1-\alpha$. Para 2 características al 95 % vale 5.991, y su raíz es 2.45.

Todos los puntos del contorno están a la misma distancia de Mahalanobis del centro. Para dibujarlo se usa la forma paramétrica, que sale de los valores y vectores propios de la covarianza:

$$\boldsymbol{\Sigma}_0 = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T, \qquad \mathbf{x}(\theta) = \boldsymbol{\mu}_0 + \sqrt{\chi^2_{p,\,1-\alpha}}\,\mathbf{V}\,\boldsymbol{\Lambda}^{1/2}\begin{bmatrix}\cos\theta \\ \sin\theta\end{bmatrix}$$

Donde:

- $\mathbf{V}$: la matriz de vectores propios, que da la dirección de los ejes de la elipse.
- $\mathbf{\Lambda}$: la matriz diagonal de valores propios, con la varianza a lo largo de cada eje.
- θ: el ángulo, de 0 a 2π.

Se toma un círculo de radio 2.45, se estira cada eje según la raíz de su valor propio, se rota con los vectores propios y se traslada al centro.

```
# Bivariate Gaussian fitted to each feature pair and scenario, and its 95 % contour:
d^2 = chi2_0.95(2)
CHI2_95 = stats.chi2.ppf(0.95, df=2)              # 5.991

def gaussian_contour(X, level=CHI2_95, n_points=200):
    mu = X.mean(axis=0)                           # mean of each feature
    cov = np.cov(X, rowvar=False, ddof=1)         # 2x2 covariance matrix
    rho = cov[0, 1] / np.sqrt(cov[0, 0] * cov[1, 1])
    # Ellipse d^2 = level: axes along the eigenvectors of cov, semi-axes sqrt(level *
eigenvalue)
    eigvals, eigvecs = np.linalg.eigh(cov)
    t = np.linspace(0, 2 * np.pi, n_points)
    circle = np.vstack([np.cos(t), np.sin(t)])
    ellipse = mu + (eigvecs @ (np.sqrt(level * eigvals)[:, None] * circle)).T
    return {'mu': mu, 'cov': cov, 'rho': rho, 'ellipse': ellipse}


# gauss[pair][scenario]: mean, covariance, rho and (n_points, 2) points of the 95 %
ellipse
pairs = {'f1-f2': {sc: freqs[sc][:, :2] for sc in SCENARIOS},
         'MAC1-MAC2': mac}
gauss = {pair: {sc: gaussian_contour(X[sc]) for sc in SCENARIOS} for pair, X in
pairs.items()}
```

```
# Scatter of each feature pair with the fitted bivariate Gaussian contours, the 95 %
ellipse and rho
from matplotlib.lines import Line2D

pair_plots = {
    'f1-f2': {'labels': ('$f_1$ (Hz)', '$f_2$ (Hz)'), 'file': 'C1_f1_f2_gaussian.png',
              'caption': 'Natural frequencies $f_1$ versus $f_2$'},
    'MAC1-MAC2': {'labels': (r'$\mathrm{MAC}_1$', r'$\mathrm{MAC}_2$'), 'file':
'C1_MAC1_MAC2_gaussian.png',
                  'caption': r'$\mathrm{MAC}_1$ versus $\mathrm{MAC}_2$'},
}

for n, (pair, cfg) in enumerate(pair_plots.items(), start=3):
    share = pair == 'f1-f2'                       # frequencies on the same axes; MAC
panels scaled to their own data
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharex=share, sharey=share)
    for ax, sc, label in zip(axes, SCENARIOS, ['(a)', '(b)', '(c)']):
        X, g = pairs[pair][sc], gauss[pair][sc]
        # Grid around the data and the ellipse to evaluate the fitted Gaussian pdf
        pts = np.vstack([X, g['ellipse']])
        lo, hi = pts.min(axis=0), pts.max(axis=0)
        pad = 0.1 * (hi - lo)
```

```
        gx, gy = np.meshgrid(np.linspace(lo[0] - pad[0], hi[0] + pad[0], 200),
                             np.linspace(lo[1] - pad[1], hi[1] + pad[1], 200))
        pdf = stats.multivariate_normal(g['mu'], g['cov']).pdf(np.dstack([gx, gy]))
        ax.contour(gx, gy, pdf, levels=6, cmap='Blues', linewidths=1)
        ax.scatter(X[:, 0], X[:, 1], s=30, alpha=0.8, color='tab:blue', zorder=3)
        ax.plot(*g['ellipse'].T, color='tab:red', lw=1.8)
        ax.plot(*g['mu'], 'k+', ms=10, mew=1.5)
        ax.text(0.03, 0.97, rf'$\rho$ = {g["rho"]:.3f}', transform=ax.transAxes,
va='top',
                bbox=dict(boxstyle='round', fc='white', alpha=0.8))
        ax.set_title(f'{label} {sc}')
        ax.set_xlabel(cfg['labels'][0])
        ax.set_ylabel(cfg['labels'][1])
        ax.xaxis.set_major_locator(plt.MaxNLocator(5))
        ax.ticklabel_format(useOffset=False)      # show the actual values, not an
offset (+1)
        ax.grid(alpha=0.3)
    handles = [Line2D([], [], ls='', marker='o', color='tab:blue', alpha=0.8,
label='Measurements'),
               Line2D([], [], color='steelblue', lw=1, label='Fitted Gaussian pdf
contours'),
               Line2D([], [], color='tab:red', lw=1.8, label=r'95% confidence ellipse
($d^2$ = 5.99)'),
               Line2D([], [], ls='', marker='+', color='k', ms=10, mew=1.5,
label='Mean')]
    fig.tight_layout()
    fig.legend(handles=handles, loc='upper center', ncol=4, bbox_to_anchor=(0.5, 0.0))
    show_figure(fig, n, f'{cfg["caption"]} with the fitted bivariate Gaussian
contours, the 95% confidence ellipse '
                r'and the correlation coefficient $\rho$ of each scenario: (a) intact,
(b) damage1, (c) damage2.',
                'part-c', cfg['file'])
```

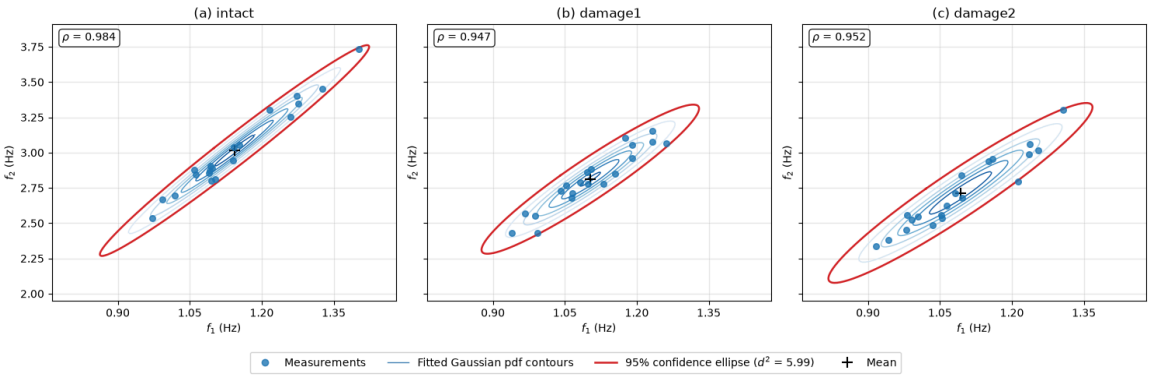


Figure 3. Natural frequencies f_1 versus f_2 with the fitted bivariate Gaussian contours, the 95% confidence ellipse and the correlation coefficient ρ of each scenario: (a) intact, (b) damage1, (c) damage2.

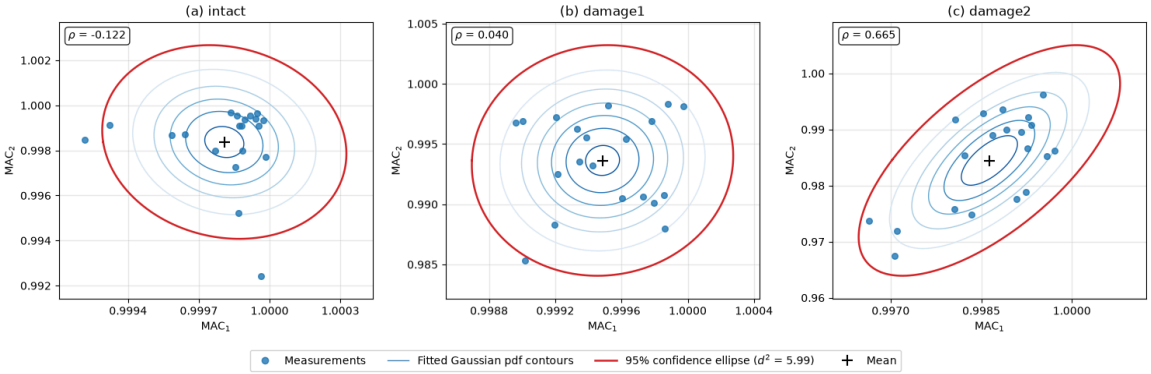


Figure 4. MAC_1 versus MAC_2 with the fitted bivariate Gaussian contours, the 95% confidence ellipse and the correlation coefficient ρ of each scenario: (a) intact, (b) damage1, (c) damage2.

Las frecuencias naturales sí están correlacionadas, como lo muestran la forma de la nube de datos en la gráfica f_1 vs f_2 y el coeficiente de correlación. Físicamente, esto se explica porque la dispersión de las mediciones de las frecuencias naturales depende principalmente de dos cosas: los cambios globales de rigidez o de masa del puente (por cambios de temperatura o de tráfico) y los errores de identificación de cada modo. La correlación mide el peso que tiene la variación común de las dos variables respecto a su variación total. En este caso, un cambio global en la rigidez o en la masa del puente tiene un impacto mayor que los errores de identificación y desplaza las frecuencias naturales de todos los modos en el mismo sentido. Por eso el coeficiente de correlación es cercano a 1.

Por otra parte, las formas modales son sensibles principalmente a cambios en la distribución de la rigidez, no a su magnitud total. Ante un daño local, esta distribución cambia, y por eso en la figura 4 el centroide de la nube de datos se desplaza por debajo de 1 tanto para MAC_1 como para MAC_2. En cambio, los cambios globales de rigidez o de masa no alteran esa distribución, por lo que casi no influyen en la dispersión de los MAC dentro de cada escenario. Esta dispersión depende principalmente de los errores de identificación, que son independientes para cada modo de vibración; por eso los MAC no están correlacionados entre sí.

La correlación de 0.665 obtenida para los MAC del daño 2 está influida en gran medida por los tres puntos de la esquina inferior izquierda. Estas mediciones deberían analizarse en detalle, porque no corresponden al comportamiento esperado según el análisis anterior.

Es importante notar que los MAC tienen físicamente un límite en 1 y que todas las elipses calculadas exceden ese límite, lo cual indica que la distribución gaussiana no es del todo apropiada para analizar estas variables.

C2. Mahalanobis Distance Detection (8 pts)

Para identificar posibles anomalías para cada par de características y cada escenario de daño, se calcula la distancia de Mahalanobis con la siguiente ecuación:

$$D = \sqrt{(\mathbf{x} - \boldsymbol{\mu}_0)^T \, \boldsymbol{\Sigma}_0^{-1} \, (\mathbf{x} - \boldsymbol{\mu}_0)}$$

- $\mathbf{x}$: el vector con las características para cada par de features en los escenarios de daño.
- $\boldsymbol{\mu}_0$: el vector de medias de cada feature de la línea base sana.

- $\mathbf{\Sigma}_0^{-1}$: la inversa de la matriz de covarianza de la línea base.

y se compara con el threshold calculado para el escenario intacto con un 95% de confiabilidad ($D_M \approx 2.45$).

```
# Mahalanobis distance of every measurement to the intact baseline (mean and
covariance of the intact scenario)
D_THRESHOLD = np.sqrt(CHI2_95)                    # 2.448: distance on the 95 % intact
ellipse

def mahalanobis(X, mu, cov):
    d = X - mu                                    # (n, 2) deviations from the baseline
mean
    return np.sqrt(np.einsum('ij,jk,ik->i', d, np.linalg.inv(cov), d))


# dm[pair][scenario]: (N_MEAS,) distance of each measurement to the intact mean
dm = {pair: {sc: mahalanobis(X[sc], gauss[pair]['intact']['mu'], gauss[pair]['intact']
['cov'])
             for sc in SCENARIOS}
      for pair, X in pairs.items()}


# Rows: feature pairs. Left: measurements and intact 95 % boundary. Right: histogram
of D_M per scenario
colors = dict(zip(SCENARIOS, ['tab:blue', 'tab:orange', 'tab:red']))
BIN_WIDTH = {'f1-f2': 0.5, 'MAC1-MAC2': 2.5}       # width of each D_M range in the
histograms
fig, axes = plt.subplots(2, 2, figsize=(13, 9.5))
for row, (pair, cfg) in enumerate(pair_plots.items()):
    ax_sc, ax_h = axes[row]
    g0 = gauss[pair]['intact']
    for sc in SCENARIOS:
        ax_sc.scatter(*pairs[pair][sc].T, s=30, alpha=0.8, color=colors[sc], label=sc)
    ax_sc.plot(*g0['ellipse'].T, color='k', lw=1.8, label=rf'Intact 95% boundary
($D_M$ = {D_THRESHOLD:.2f})')
    ax_sc.plot(*g0['mu'], 'k+', ms=10, mew=1.5, label='Intact mean')
    ax_sc.set_title(f'({"ac"[row]}) {cfg["caption"]}')
    ax_sc.set_xlabel(cfg['labels'][0])
    ax_sc.set_ylabel(cfg['labels'][1])
    ax_sc.xaxis.set_major_locator(plt.MaxNLocator(5))
    ax_sc.ticklabel_format(useOffset=False)
    ax_sc.grid(alpha=0.3)
    ax_sc.legend(fontsize=8)

    d_max = max(dm[pair][sc].max() for sc in SCENARIOS)
    bins = np.arange(0, d_max + BIN_WIDTH[pair], BIN_WIDTH[pair])
    for sc in SCENARIOS:
        ax_h.hist(dm[pair][sc], bins=bins, alpha=0.5, color=colors[sc],
edgecolor=colors[sc], label=sc)
    ax_h.axvline(D_THRESHOLD, color='k', ls='--', lw=1.5, label=f'Threshold ($D_M$ =
{D_THRESHOLD:.2f})')
    ax_h.set_title(f'({"bd"[row]}) Mahalanobis distance, {pair}')
    ax_h.set_xlabel('$D_M$ to the intact baseline')
    ax_h.set_ylabel('Number of measurements')
```

```
    ax_h.grid(alpha=0.3)
    ax_h.legend(fontsize=8)
fig.tight_layout()
show_figure(fig, 5, 'Mahalanobis distance detection with the intact scenario as
baseline: (a) $f_1$ versus $f_2$ and '
            r'(c) $\mathrm{MAC}_1$ versus $\mathrm{MAC}_2$ of the 20 measurements of
each scenario with the intact 95% '
            r'boundary; (b) and (d) histograms of the Mahalanobis distance $D_M$ of
each measurement to the intact mean, '
            'with the 95% threshold.', 'part-c', 'C2_mahalanobis.png')
```

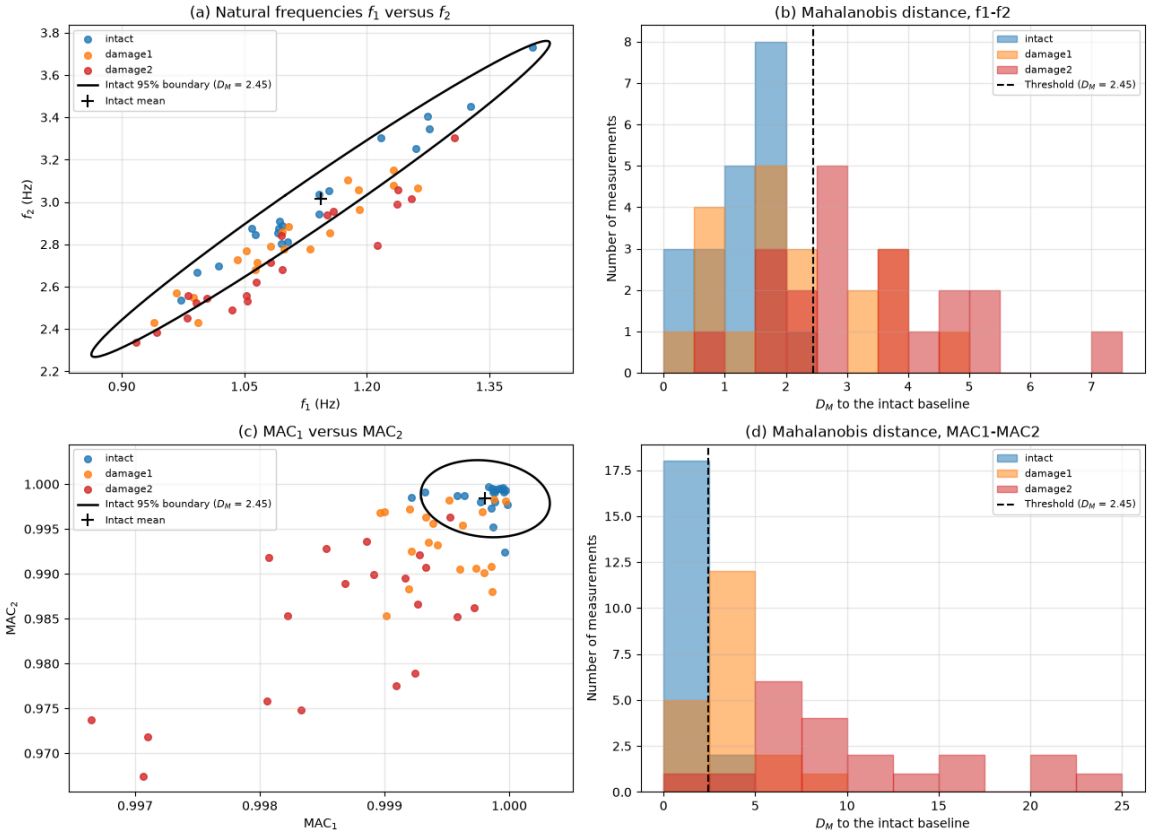


Figure 5. Mahalanobis distance detection with the intact scenario as baseline: (a) f_1 versus f_2 and (c) MAC_1 versus MAC_2 of the 20 measurements of each scenario with the intact 95% boundary; (b) and (d) histograms of the Mahalanobis distance D_M of each measurement to the intact mean, with the 95% threshold.

Para reportar los resultados se calcula, para cada par de características y cada escenario, el porcentaje de mediciones que superan el umbral:

$$\text{Rate} = \frac{\text{Number of measurements with } D_M > 2.45}{\text{Total number of measurements in the scenario}} \times 100\% \tag{2}$$

El mismo cálculo se interpreta distinto según el escenario:

- **damage1 y damage2:** es la tasa de detección (detection rate), el porcentaje del daño que se detecta correctamente.
- **intact:** es la tasa de falsas alarmas (false alarm rate), el porcentaje del estado sano que se marca como dañado. Con un umbral del 95 % se espera alrededor del 5 %.

```
# Percentage of measurements above the threshold (Eq. 2): false alarms (intact) and
detections (damage)
def rate_text(d):
    k, n = (d > D_THRESHOLD).sum(), len(d)  # measurements above the threshold, total
measurements
    return f'{100 * k / n:.0f}% ({k}/{n})'


rates = pd.DataFrame({sc: {pair: rate_text(dm[pair][sc]) for pair in dm} for sc in
SCENARIOS})
rates.index.name = 'Feature pair'
rates.columns = ['False alarm rate, intact', 'Detection rate, damage1', 'Detection
rate, damage2']
show_table(rates, 9, r'Percentage of measurements with $D_M$ above the 95% threshold
($D_M > 2.45$) '
           'for each feature pair and scenario.', 'C2_detection_rates.csv',
precision=1)
```

Table 9. Percentage of measurements with D_M above the 95% threshold ($D_M > 2.45$) for each feature pair and scenario.

Feature pair	False alarm rate, intact	Detection rate, damage1	Detection rate, damage2
f1-f2	0% (0/20)	30% (6/20)	70% (14/20)
MAC1-MAC2	10% (2/20)	75% (15/20)	95% (19/20)

C3. Bivariate vs Univariate Comparison (5 pts)

```
# Univariate z-score results from Part B (|z| > 2): measurements flagged out of 20 per
scenario
# PROVISIONAL: assumed values, replace with the Part B results once B1 is done
z_flagged = {'f1':   {'intact': 1, 'damage1': 0,  'damage2': 1},
             'f2':   {'intact': 1, 'damage1': 0,  'damage2': 2},
             'MAC1': {'intact': 2, 'damage1': 9,  'damage2': 17},
             'MAC2': {'intact': 1, 'damage1': 10, 'damage2': 19}}
N_MEAS = 20  # measurements per scenario

# One table per feature pair: individual features (z-score) next to the pair
(Mahalanobis, Table 9)
pair_tables = {'f1-f2':     (10, 'natural frequencies',
'C3_comparison_frequencies.csv'),
               'MAC1-MAC2': (11, 'MAC values', 'C3_comparison_MAC.csv')}
for pair, (number, name, filename) in pair_tables.items():
    rows = []
    for feat in pair.split('-'):
        rows.append([feat, 'z-score, |z| > 2']
                    + [f'{100 * z_flagged[feat][sc] / N_MEAS:.0f}% ({z_flagged[feat]
[sc]}/{N_MEAS})' for sc in SCENARIOS])
    rows.append([pair, r'Mahalanobis, $D_M$ > 2.45'] + list(rates.loc[pair]))
    comparison = pd.DataFrame(rows, columns=['Feature(s)', 'Method'] +
list(rates.columns)).set_index('Feature(s)')
    show_table(comparison, number, f'Detection performance of the individual {name}
(univariate z-score, Part B) '
```

```
            f'versus the {pair} pair (bivariate Mahalanobis distance, Table 9).',
filename)
```

Table 10. Detection performance of the individual natural frequencies (univariate z-score, Part B) versus the f1-f2 pair (bivariate Mahalanobis distance, Table 9).

Feature(s)	Method	False alarm rate, intact	Detection rate, damage1	Detection rate, damage2
f1	z-score, |z| > 2	5% (1/20)	0% (0/20)	5% (1/20)
f2	z-score, |z| > 2	5% (1/20)	0% (0/20)	10% (2/20)
f1-f2	Mahalanobis, D_M > 2.45	0% (0/20)	30% (6/20)	70% (14/20)

Table 11. Detection performance of the individual MAC values (univariate z-score, Part B) versus the MAC1-MAC2 pair (bivariate Mahalanobis distance, Table 9).

Feature(s)	Method	False alarm rate, intact	Detection rate, damage1	Detection rate, damage2
MAC1	z-score, |z| > 2	10% (2/20)	45% (9/20)	85% (17/20)
MAC2	z-score, |z| > 2	5% (1/20)	50% (10/20)	95% (19/20)
MAC1-MAC2	Mahalanobis, D_M > 2.45	10% (2/20)	75% (15/20)	95% (19/20)

Pendiente confirmar con los datos reales de la parte B.

Cuando dos variables están correlacionadas, el análisis bivariado es más efectivo para detectar desplazamientos en la mayoría de las direcciones, sobre todo en contra de la correlación, porque toma en cuenta la densidad conjunta de las dos variables a través de la distancia de Mahalanobis. De esta manera, un dato que se desplaza en contra de la correlación, o en el que solo una de las variables se mueve, es mucho más raro que si se analiza cada variable por separado. Si se analizan por separado, cada variable puede desplazarse una distancia que no necesariamente excede el umbral del 95 %, pero la distribución bivariada le da un mayor peso a esa variación cuando no sigue la correlación de los datos sanos.

Por el contrario, cuando las dos variables se desplazan en la misma dirección de su correlación, el análisis bivariado les da un menor peso, porque ese comportamiento es normal en los datos sanos. Esto ocurre con los cambios globales, como la temperatura o la masa del tráfico, que cambian la rigidez o la masa de todo el puente y desplazan las frecuencias de todos los modos en la misma proporción.

Sin embargo, el objetivo es detectar anomalías producto de daños, no de cambios de temperatura ni de masa. Cuando surge un daño, el daño es local y, pese a que disminuye la rigidez total del puente, cada modo se ve afectado en una proporción diferente. Por ende, la relación entre las frecuencias naturales de cada modo cambia, y la medición se desplaza en

una dirección diferente a la correlación del estado intacto. Ese es el caso en el que el análisis bivariado es mucho más efectivo que el análisis de las variables por separado.

En el caso de los MAC, pese a no estar correlacionados, el análisis bivariado igual es más efectivo, porque suma la evidencia de las dos variables: dos desviaciones moderadas que por separado no exceden el umbral pueden excederlo cuando se analizan juntas. Lo único es que, en los MAC, quizás ese contorno no representa realmente el percentil del 95 %, sino un porcentaje diferente. Pero igual sigue siendo efectivo para la detección de anomalías respecto a la detección con las variables separadas.

Los resultados confirman lo anterior. En las frecuencias naturales, el análisis por separado prácticamente no detecta el daño: f_1 y f_2 detectan 0 % en el daño 1 y entre 5 % y 10 % en el daño 2. En cambio, el par (f_1, f_2) con la distancia de Mahalanobis detecta 30 % y 70 %, respectivamente, sin falsas alarmas en el estado intacto. Esto indica que, aunque parte del desplazamiento ocurre en la dirección de la correlación, el daño sí saca las mediciones de esa dirección. En los MAC, el análisis bivariado sube la detección del daño 1 de 45–50 % a 75 %, y en el daño 2 iguala a la mejor variable individual (95 %), con la misma tasa de falsas alarmas que MAC_1 (10 %). Por lo tanto, tomar en cuenta la correlación sí mejora la detección.

Entre los dos pares, los MAC son más efectivos para la detección del daño, con tasas de 75 % y 95 % frente a 30 % y 70 % de las frecuencias. Esto se debe a que el daño local cambia la distribución de la rigidez, que es justo lo que miden las formas modales, mientras que las frecuencias también cambian con la temperatura y la masa. Sin embargo, el par de MAC tiene una tasa de falsas alarmas de 10 % en el estado intacto, mayor que el 5 % esperado, lo que coincide con que la distribución gaussiana no es del todo apropiada para estas variables.

Part D: Feature Fusion and Engineering (15 pts)

D1. Systematic Combination Testing (10 pts)

Se genera el diccionario `features` con las matrices de las 6 features para cada escenario (3 escenarios x 20 muestras x 6 features).

```
# All six features in one matrix per scenario: [f1, f2, f3, MAC1, MAC2, MAC3]
FEATURES = ['f1', 'f2', 'f3', 'MAC1', 'MAC2', 'MAC3']

# MAC (Eq. 1) of the three modes against the mean intact mode shape
mac_all = {sc: np.array([[calculate_MAC(ref_shapes[j], shapes[sc][j, i]) for j in
range(len(MODES))]
                         for i in range(N_MEAS)])
           for sc in SCENARIOS}

# features[scenario]: (N_MEAS, 6) array -> measurements x features
features = {sc: np.column_stack([freqs[sc], mac_all[sc]]) for sc in SCENARIOS}
```

Se estandarizan los valores tomando como linea base las mediciones del escenario intacto con la siguiente ecuación:

$$z_{ij} = \frac{x_{ij} - \mu_{0,j}}{\sigma_{0,j}} \tag{3}$$

donde:

- x_{ij}: valor de la feature j en la medición i (de cualquier escenario).
- $\mu_{0,j}$: media de la feature j en el escenario intacto.
- $\sigma_{0,j}$: desviación estándar muestral (ddof = 1) de la feature j en el escenario intacto.
- z_{ij}: valor estandarizado.

```
# Step 1: mean and standard deviation of each feature in the intact scenario
(baseline)
mu0 = features['intact'].mean(axis=0)            # (6,) mean of each feature
sigma0 = features['intact'].std(axis=0, ddof=1)  # (6,) sample standard deviation of
each feature

baseline_table = pd.DataFrame({'Mean': mu0, 'Standard deviation': sigma0},
                              index=pd.Index(FEATURES, name='Feature'))
show_table(baseline_table, 12, 'Mean and standard deviation of each feature in the
intact scenario '
           '(baseline used for the standardization).', 'D1_intact_baseline.csv',
precision=6)
```

Table 12. Mean and standard deviation of each feature in the intact scenario (baseline used for the standardization).

Feature	Mean	Standard deviation
f1	1.143267	0.114653
f2	3.014206	0.305121
f3	4.347781	0.448803
MAC1	0.999808	0.000211
MAC2	0.998374	0.001756
MAC3	0.998017	0.001389

```
# Step 2: standardize every scenario with the intact statistics: z = (x - mu0) /
sigma0
features_std = {sc: (features[sc] - mu0) / sigma0 for sc in SCENARIOS}

# Check: the standardized intact features must have mean 0 and standard deviation 1
assert np.allclose(features_std['intact'].mean(axis=0), 0)
assert np.allclose(features_std['intact'].std(axis=0, ddof=1), 1)
for sc in SCENARIOS:
    print(f'{sc}: mean = {np.round(features_std[sc].mean(axis=0), 2)}, '
          f'std = {np.round(features_std[sc].std(axis=0, ddof=1), 2)}')
```

```
intact: mean = [-0. -0. -0. -0.  0. -0.], std = [1. 1. 1. 1. 1. 1.]
damage1: mean = [-0.35 -0.66 -0.35 -1.52 -2.7  -3.15], std = [0.81 0.71 0.74 1.53 2.23
2.82]
damage2: mean = [ -0.44  -0.98  -0.4   -5.55  -7.92 -10.74], std = [0.98 0.85 0.83
4.18 4.78 6.43]
```

Se generan todas las posibles combinaciones de 4 de 6 features que se pueden tener y se calcula la distancia de Wasserstein tanto para el escenario de daño 1 como el escenario de

daño 2 tomando como referencia el escenario intacto.

```
# All ways to select 4 of the 6 features: C(6, 4) = 15 combinations (column indices in features_std)
from itertools import combinations

combos = list(combinations(range(len(FEATURES)), 4))
combo_names = [', '.join(FEATURES[j] for j in c) for c in combos]   # e.g. 'f1, f2, f3, MAC1'
assert len(combos) == 15

# Multivariate Wasserstein distance between intact and each damage scenario, for each 4-feature combination
DAMAGES = ['damage1', 'damage2']

# w_combo[damage]: (15,) distance of each combination, computed on the standardized features
w_combo = {dmg: np.array([stats.wasserstein_distance_nd(features_std['intact'][:, list(c)],
                                                        features_std[dmg][:, list(c)])
                          for c in combos])
           for dmg in DAMAGES}

w_combo_table = pd.DataFrame({'Features': combo_names,
                              **{f'W (intact, {dmg})': w_combo[dmg] for dmg in DAMAGES}},
                             index=pd.RangeIndex(1, len(combos) + 1, name='Combination'))
show_table(w_combo_table, 13, 'Multivariate Wasserstein distance between the intact scenario and each damage '
           'scenario for the 15 combinations of 4 standardized features.', 'D1_wasserstein_combinations.csv',
           precision=3)
```

Table 13. Multivariate Wasserstein distance between the intact scenario and each damage scenario for the 15 combinations of 4 standardized features.

Combination	Features	W (intact, damage1)	W (intact, damage2)
1	f1, f2, f3, MAC1	1.872	5.704
2	f1, f2, f3, MAC2	2.903	8.024
3	f1, f2, f3, MAC3	3.335	10.822
4	f1, f2, MAC1, MAC2	3.417	9.925
5	f1, f2, MAC1, MAC3	3.801	12.382
6	f1, f2, MAC2, MAC3	4.291	13.432
7	f1, f3, MAC1, MAC2	3.362	9.886
8	f1, f3, MAC1, MAC3	3.751	12.348
9	f1, f3, MAC2, MAC3	4.242	13.400

Combination	Features	W (intact, damage1)	W (intact, damage2)
10	f1, MAC1, MAC2, MAC3	4.623	14.674
11	f2, f3, MAC1, MAC2	3.421	9.925
12	f2, f3, MAC1, MAC3	3.815	12.383
13	f2, f3, MAC2, MAC3	4.297	13.430
14	f2, MAC1, MAC2, MAC3	4.660	14.702
15	f3, MAC1, MAC2, MAC3	4.623	14.674

```
# Bar chart of the Wasserstein distance of each combination, one panel per damage
scenario; top 3 highlighted
from matplotlib.patches import Patch

top3 = {dmg: np.argsort(w_combo[dmg])[::-1][:3] for dmg in DAMAGES}   # indices of the
3 largest distances

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5))
x = np.arange(len(combos))
for ax, dmg, label in zip(axes, DAMAGES, ['(a)', '(b)']):
    bar_colors = ['tab:red' if k in top3[dmg] else 'tab:gray' for k in x]
    ax.bar(x, w_combo[dmg], color=bar_colors, alpha=0.85)
    for rank, k in enumerate(top3[dmg], start=1):
        ax.text(k, w_combo[dmg][k], f'#{rank}', ha='center', va='bottom', fontsize=9,
fontweight='bold')
    ax.set_xticks(x, combo_names, rotation=75, ha='right', rotation_mode='anchor')
    ax.set_title(f'{label} Intact vs {dmg}')
    ax.set_xlabel('Feature combination')
    ax.set_ylabel('Multivariate Wasserstein distance')
    ax.grid(axis='y', alpha=0.3)
    ax.legend(handles=[Patch(color='tab:red', alpha=0.85, label='Top 3 combinations'),
                       Patch(color='tab:gray', alpha=0.85, label='Other
combinations')], loc='upper left')
fig.tight_layout()
show_figure(fig, 6, 'Multivariate Wasserstein distance between the intact scenario and
(a) damage1, (b) damage2 for the '
            '15 combinations of 4 standardized features. The 3 most sensitive
combinations of each scenario are '
            'highlighted and ranked.', 'part-d', 'D1_wasserstein_combinations.png')
```

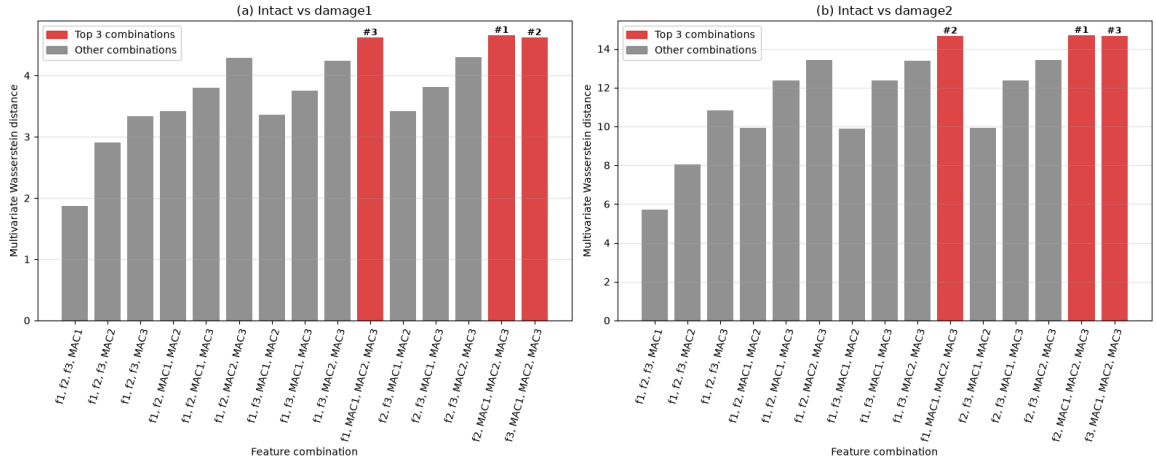


Figure 6. Multivariate Wasserstein distance between the intact scenario and (a) damage1, (b) damage2 for the 15 combinations of 4 standardized features. The 3 most sensitive combinations of each scenario are highlighted and ranked.

```
# Top 3 most sensitive combinations (largest Wasserstein distance) for each damage
scenario
top3_table = pd.DataFrame(
    {(dmg, col): [val(k) for k in top3[dmg]]
     for dmg in DAMAGES
     for col, val in [('Features', lambda k: combo_names[k]), ('W', lambda k:
w_combo[dmg][k])]},
    index=pd.RangeIndex(1, 4, name='Rank'))
show_table(top3_table, 14, 'Top 3 most sensitive combinations of 4 features (largest
multivariate Wasserstein '
           'distance to the intact scenario) for each damage scenario.',
'D1_top3_combinations.csv', precision=3)
```

Table 14. Top 3 most sensitive combinations of 4 features (largest multivariate Wasserstein distance to the intact scenario) for each damage scenario.

	damage1		damage2	
	Features	W	Features	W
Rank				
1	f2, MAC1, MAC2, MAC3	4.660	f2, MAC1, MAC2, MAC3	14.702
2	f3, MAC1, MAC2, MAC3	4.623	f1, MAC1, MAC2, MAC3	14.674
3	f1, MAC1, MAC2, MAC3	4.623	f3, MAC1, MAC2, MAC3	14.674

D2. Feature Importance Analysis (5 pts)

Pendiente confirmar con los datos reales de la parte B.

Las características que más se repiten en las combinaciones con mejor desempeño son los MAC.

Las combinaciones dominadas por las frecuencias tienen una distancia de Wasserstein entre 1.872 y 3.335 para el escenario de daño 1 y entre 5.704 y 10.822 para el escenario de daño 2. Mientras que las combinaciones dominadas por los MAC tienen una distancia de Wasserstein

aproximada de 4.6 para el escenario de daño 1 y de aproximadamente 14.7 para el escenario de daño 2. Esto muestra que las combinaciones dominadas por los MAC son más sensibles al daño que las dominadas por las frecuencias. Este resultado confirma lo encontrado en la parte B2, donde los MAC resultaron ser las características más sensibles al daño.

```
# Normalized Wasserstein distance of each single feature from Part B2 (W1 / sigma0)
# PROVISIONAL: assumed values, replace with the Part B results once B2 is done
w1_norm = {'damage1': dict(zip(FEATURES, [0.361332, 0.682204, 0.365825, 1.565752,
2.681335, 3.174288])),
           'damage2': dict(zip(FEATURES, [0.452566, 1.009418, 0.415180, 5.695773,
7.898526, 10.871832]))}

# Best 4 single features (Part B2) next to the best 4 combinations of 4 features (D1,
Table 13) for each damage
top4_feat = {dmg: sorted(w1_norm[dmg], key=w1_norm[dmg].get, reverse=True)[:4] for dmg
in DAMAGES}
top4_combo = {dmg: np.argsort(w_combo[dmg])[::-1][:4] for dmg in DAMAGES}

best_table = pd.DataFrame(
    {(dmg, col): vals
     for dmg in DAMAGES
     for col, vals in [('Feature (B2)', top4_feat[dmg]),
                       ('W1,norm', [w1_norm[dmg][f] for f in top4_feat[dmg]]),
                       ('Combination (D1)', [combo_names[k] for k in
top4_combo[dmg]]),
                       ('W', w_combo[dmg][top4_combo[dmg]])]},
    index=pd.RangeIndex(1, 5, name='Rank'))
show_table(best_table, 15, 'Best 4 single features (normalized Wasserstein distance,
Part B2) and best 4 combinations of '
           '4 features (multivariate Wasserstein distance, Table 13) for each damage
scenario.',
           'D2_best_features_vs_combinations.csv', precision=3)
```

Table 15. Best 4 single features (normalized Wasserstein distance, Part B2) and best 4 combinations of 4 features (multivariate Wasserstein distance, Table 13) for each damage scenario.

	damage1				damage2			
	Feature (B2)	W1,norm	Combination (D1)	W	Feature (B2)	W1,norm	Combination (D1)	W
Rank								
1	MAC3	3.174	f2, MAC1, MAC2, MAC3	4.660	MAC3	10.872	f2, MAC1, MAC2, MAC3	14.702
2	MAC2	2.681	f3, MAC1, MAC2, MAC3	4.623	MAC2	7.899	f1, MAC1, MAC2, MAC3	14.674
3	MAC1	1.566	f1, MAC1, MAC2, MAC3	4.623	MAC1	5.696	f3, MAC1, MAC2, MAC3	14.674
4	f2	0.682	f2, f3, MAC2, MAC3	4.297	f2	1.009	f1, f2, MAC2, MAC3	13.432

Pendiente por confirmar con los datos reales de la parte B.

De la tabla 15 se puede observar que la combinación de las 4 primeras características obtenidas en el análisis de la parte B es la combinación más sensible al daño para ambos escenarios de daño.

La mejor combinación (f_2, MAC_1, MAC_2, MAC_3) tiene una distancia de Wasserstein de 4.660 para el escenario de daño 1 y de 14.702 para el escenario de daño 2, mientras que la mejor característica individual, MAC_3, tiene 3.174 y 10.872, respectivamente. Esto representa un aumento de aproximadamente 47 % y 35 %. Dado que la distancia de Wasserstein compara qué tan desplazadas están dos distribuciones, la fusión de características sí aumenta la sensibilidad al daño. Sin embargo, parte de este aumento se debe solo a que se suman más dimensiones, por lo que esta distancia indica una mayor separación entre las distribuciones, pero no necesariamente una mejor detección de cada medición.

En el caso de detección de daños, donde se comparan distribuciones actuales con las distribuciones del escenario intacto o puente sano, tanto las frecuencias como los MAC aportan información independiente sobre el daño, porque al tener un daño local, las formas modales y la relación entre las frecuencias naturales de cada modo de vibración se ven afectadas de diferentes maneras.

Part E: Principal Component Analysis (15 pts)

E1. PCA Implementation and Variance Analysis (4 pts)

```
# The matrix features_std calculated above is used in this section
from sklearn.decomposition import PCA

# PCA fitted on the standardized intact data only (20 x 6); all 6 components are kept
pca = PCA(n_components=len(FEATURES))
pca.fit(features_std['intact'])

# Scree plot: variance explained by each PC (bars) and cumulative variance (line),
with the 95 % level
var_pct = 100 * pca.explained_variance_ratio_
cum_pct = np.cumsum(var_pct)
pcs = np.arange(1, len(var_pct) + 1)
k95 = np.argmax(cum_pct >= 95) + 1               # number of PCs needed to reach 95 %
of the variance

# (a) variance explained by each PC, (b) cumulative variance explained with the 95 %
level
fig, (ax_a, ax_b) = plt.subplots(1, 2, figsize=(13, 4.8))
ax_a.bar(pcs, var_pct, color='tab:blue', alpha=0.85)
for p, v in zip(pcs, var_pct):
    ax_a.text(p, v + 1, f'{v:.1f}%', ha='center', fontsize=9)
ax_a.set_ylim(0, 1.15 * var_pct.max())
ax_a.set_title('(a) Variance explained by each PC')
ax_a.set_ylabel('Variance explained (%)')

ax_b.plot(pcs, cum_pct, 'o-', color='tab:red', label='Cumulative variance explained')
ax_b.axhline(95, color='k', ls='--', lw=1.2, label='95% of the variance')
for p, c in zip(pcs, cum_pct):
```

```
    ax_b.text(p, c + 2, f'{c:.1f}%', ha='center', va='bottom', fontsize=9,
color='tab:red')
ax_b.set_ylim(0, 110)
ax_b.set_xlim(0.5, len(pcs) + 0.5)
ax_b.set_title('(b) Cumulative variance explained')
ax_b.set_ylabel('Cumulative variance explained (%)')
ax_b.legend(loc='lower right')

for ax in (ax_a, ax_b):
    ax.set_xticks(pcs, [f'PC{p}' for p in pcs])
    ax.set_xlabel('Principal component')
    ax.grid(axis='y', alpha=0.3)
fig.tight_layout()
show_figure(fig, 7, 'Scree plot of the PCA fitted on the 6 standardized features of
the intact scenario: (a) variance '
            f'explained by each principal component and (b) cumulative variance. {k95}
components are needed to capture '
            '95% of the variance.', 'part-e', 'E1_scree_plot.png')
```

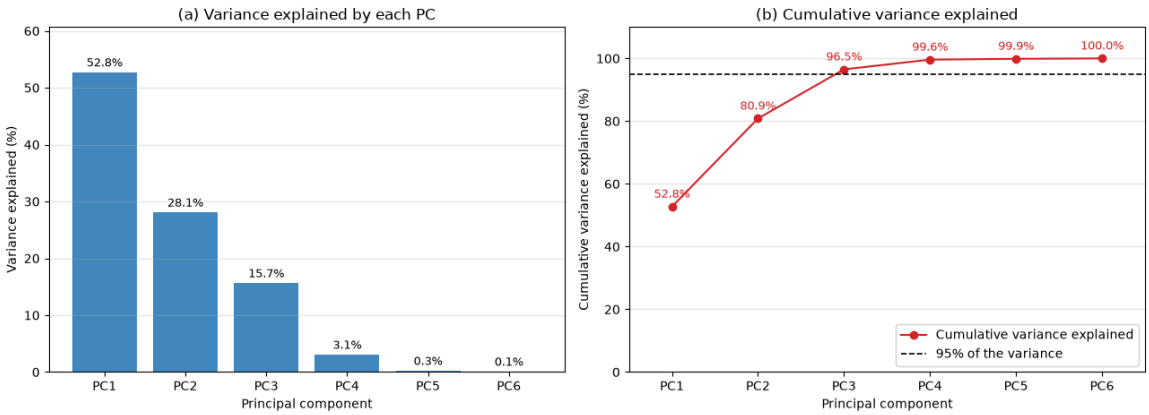


Figure 7. Scree plot of the PCA fitted on the 6 standardized features of the intact scenario: (a) variance explained by each principal component and (b) cumulative variance. 3 components are needed to capture 95% of the variance.

Como se puede observar en la figura 7, se necesitan al menos los primeros 3 componentes para representar el 95% de la varianza de los datos.

E2. First Principal Component Analysis (3 pts)

Se proyectan los datos de las 6 variables sobre la componente 1 del PCA para cada uno de los escenarios.

```
# Project the standardized features of every scenario onto PC1 only (PCA fitted on the
intact data)
# pc1[scenario]: (N_MEAS,) score of each measurement on the first principal component
pc1 = {sc: pca.transform(features_std[sc])[:, 0] for sc in SCENARIOS}

for sc in SCENARIOS:
    print(f'{sc}: {pc1[sc].shape[0]} PC1 scores, mean = {pc1[sc].mean():.3f}, std =
{pc1[sc].std(ddof=1):.3f}')
```

```
intact: 20 PC1 scores, mean = 0.000, std = 1.779
damage1: 20 PC1 scores, mean = 0.231, std = 1.503
damage2: 20 PC1 scores, mean = 1.970, std = 2.512
```

Es importante notar que, aunque los datos se estandarizaron con el escenario intacto ($\mu = 0$ y $\sigma = 1$ para cada característica), la varianza total es 6, es decir, la suma de las varianzas de las seis características. Como el componente 1 del PCA representa el 52.8 % de esa varianza total, su varianza es $0.528 \times 6 \approx 3.17$ y su desviación estándar es $\sqrt{3.17} \approx 1.78$. Por eso la desviación estándar de PC1 es mayor que 1: concentra la variación que comparten las características correlacionadas.

```
# Overlaid histograms of the PC1 scores of the three scenarios, each with its fitted
normal PDF
all_pc1 = np.concatenate([pc1[sc] for sc in SCENARIOS])
bins = np.linspace(all_pc1.min(), all_pc1.max(), 13)   # same bins for the three
scenarios
x_pdf = np.linspace(all_pc1.min() - 1, all_pc1.max() + 1, 300)

fig, ax = plt.subplots(figsize=(9, 5))
for sc in SCENARIOS:
    mu, sd = pc1[sc].mean(), pc1[sc].std(ddof=1)
    ax.hist(pc1[sc], bins=bins, density=True, alpha=0.12, color=colors[sc])
    ax.hist(pc1[sc], bins=bins, density=True, histtype='step', lw=1.5,
color=colors[sc])
    ax.plot(x_pdf, stats.norm.pdf(x_pdf, mu, sd), color=colors[sc], lw=2,
            label=rf'{sc}: $\mu$ = {mu:.2f}, $\sigma$ = {sd:.2f}')
ax.set_title('Distribution of the PC1 scores for each scenario')
ax.set_xlabel('PC1 score')
ax.set_ylabel('Probability density')
ax.grid(alpha=0.3)
ax.legend(title='Histogram and fitted normal PDF')
fig.tight_layout()
show_figure(fig, 8, 'Overlaid histograms of the PC1 scores of the intact, damage1 and
damage2 scenarios, with the normal '
            r'PDF fitted to each scenario (mean $\mu$ and standard deviation $\sigma$
in the legend). PCA fitted on the '
            'standardized intact data.', 'part-e', 'E2_PC1_distributions.png')
```

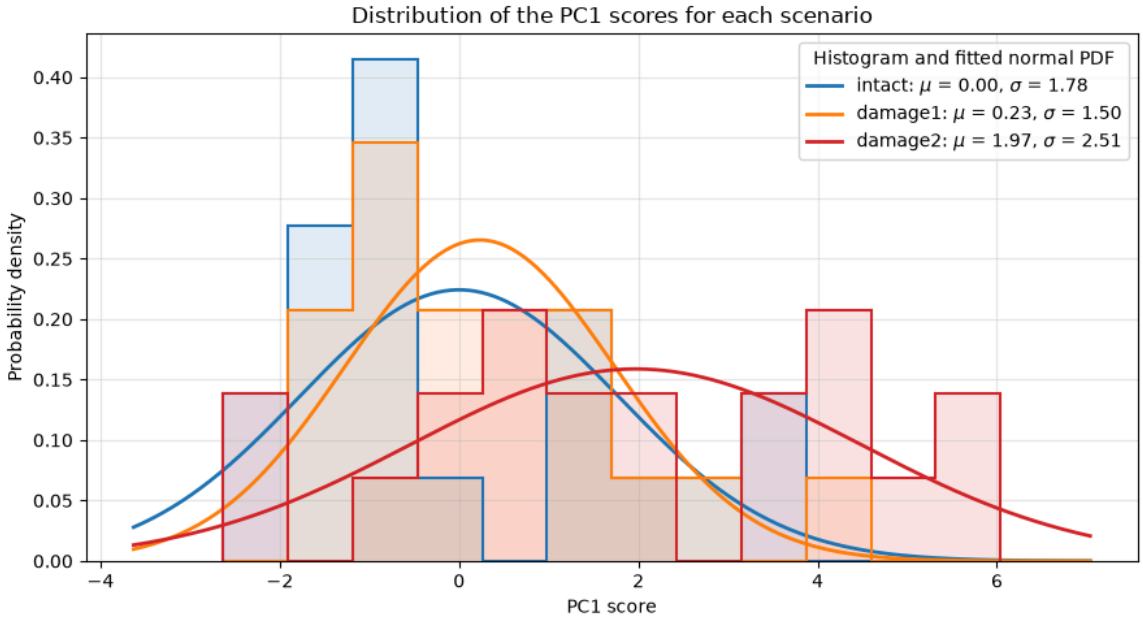


Figure 8. Overlaid histograms of the PC1 scores of the intact, damage1 and damage2 scenarios, with the normal PDF fitted to each scenario (mean μ and standard deviation σ in the legend). PCA fitted on the standardized intact data.

```
# 1D Wasserstein distance between the PC1 scores of the intact scenario and each
damage scenario
w_pc1 = {dmg: stats.wasserstein_distance(pc1['intact'], pc1[dmg]) for dmg in DAMAGES}

# Normalized as in Part B2 (W1 / sigma0): divide by the standard deviation of the
intact PC1 scores
# ddof must match the one used for sd0 in Part B
sigma0_pc1 = pc1['intact'].std(ddof=1)
w_pc1_norm = {dmg: w_pc1[dmg] / sigma0_pc1 for dmg in DAMAGES}

w_pc1_table = pd.DataFrame({f'W1,norm (intact, {dmg})': [w_pc1_norm[dmg]] for dmg in
DAMAGES},
                           index=pd.Index(['PC1'], name='PCA space'))
show_table(w_pc1_table, 16, 'Normalized Wasserstein distance $W_1/\\sigma_0$ between
the PC1 scores of the '
           'intact scenario and each damage scenario (same normalization as Part
B2).',
           'E2_wasserstein_PC1.csv', precision=3)
```

Table 16. Normalized Wasserstein distance W_1/σ_0 between the PC1 scores of the intact scenario and each damage scenario (same normalization as Part B2).

PCA space	W1,norm (intact, damage1)	W1,norm (intact, damage2)
PC1	0.269	1.107

PENDING: Compare with Part B results: is PC1 more sensitive than any single feature?

Pendiente confirmar con los resultados reales de la parte B

La distancia de Wasserstein de la PC1, normalizada respecto al escenario intacto, es menor que la de todas las características individuales en el escenario de daño 1, y en el escenario de daño 2 solo supera a las frecuencias naturales, quedando muy por debajo de los MAC. Esto se debe principalmente a que la PC1 conserva la dirección en la que más varían los datos del escenario intacto, que es la de las frecuencias naturales, porque los cambios de temperatura y de masa las desplazan juntas. Esa dirección es la menos útil para detectar el daño, mientras que el cambio en los MAC queda en otras componentes. Por eso, aunque la PC1 representa el 52.8 % de la varianza, se está perdiendo la información más importante sobre el daño.

E3. Two-Dimensional PCA Space (3 pts)

```
# Project the standardized features of every scenario onto PC1 and PC2 (PCA fitted on
the intact data)
# pc12[scenario]: (N_MEAS, 2) array -> measurements x [PC1, PC2]
pc12 = {sc: pca.transform(features_std[sc])[:, :2] for sc in SCENARIOS}

# Bivariate Gaussian fitted to the PC1-PC2 scores of each scenario and its 95 %
contour (gaussian_contour from C1)
gauss_pc = {sc: gaussian_contour(pc12[sc]) for sc in SCENARIOS}

# Scatter of PC1 versus PC2 with the 95 % ellipse and the mean of each scenario
fig, ax = plt.subplots(figsize=(8, 6))
for sc in SCENARIOS:
    ax.scatter(*pc12[sc].T, s=35, alpha=0.8, color=colors[sc], label=sc)
    ax.plot(*gauss_pc[sc]['ellipse'].T, color=colors[sc], lw=1.8, label=f'{sc} 95%
ellipse')
    ax.plot(*gauss_pc[sc]['mu'], '+', color=colors[sc], ms=12, mew=2)
ax.set_title('PC1 versus PC2 scores for each scenario')
ax.set_xlabel('PC1 score')
ax.set_ylabel('PC2 score')
ax.grid(alpha=0.3)
ax.legend(fontsize=9)
fig.tight_layout()
show_figure(fig, 9, 'Scores of the 20 measurements of each scenario on the first two
principal components (PC1 versus '
            'PC2), with the 95% confidence ellipse of the bivariate Gaussian fitted to
each scenario (+: mean). '
            'PCA fitted on the standardized intact data.', 'part-e',
'E3_PC1_PC2_scatter.png')
```

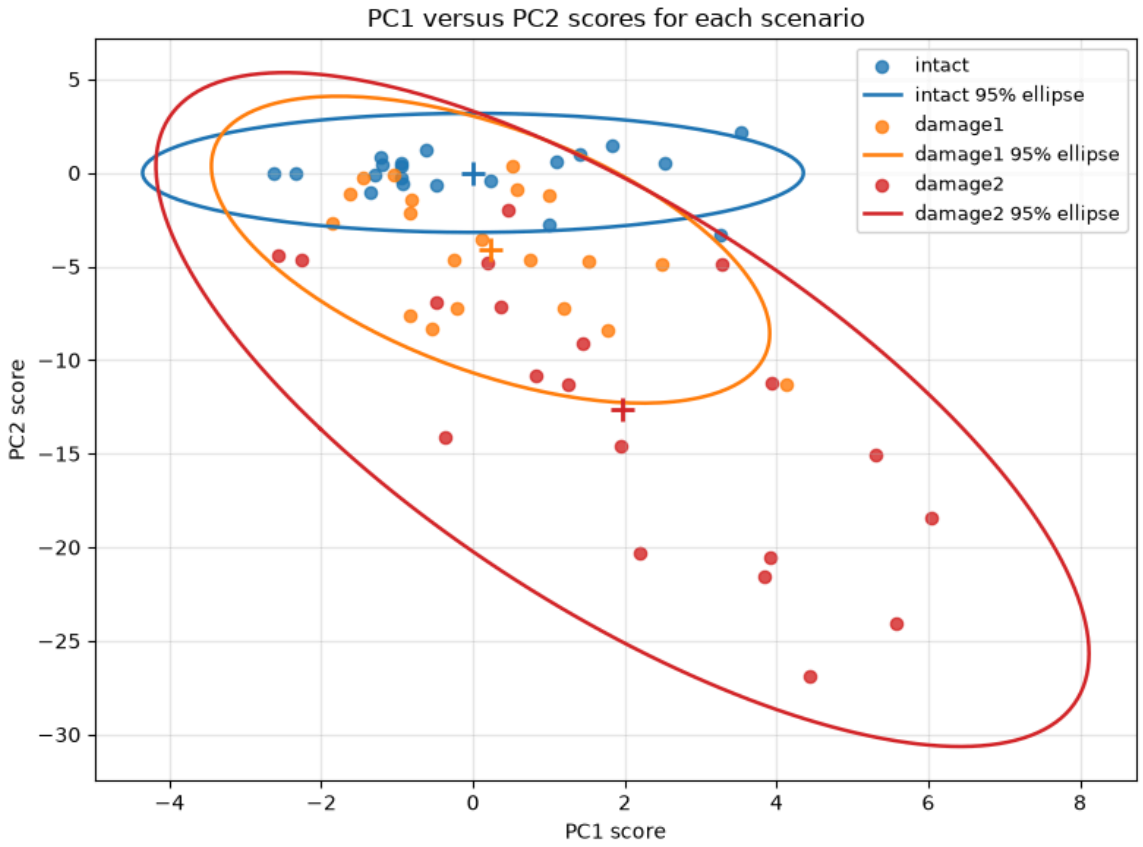


Figure 9. Scores of the 20 measurements of each scenario on the first two principal components (PC1 versus PC2), with the 95% confidence ellipse of the bivariate Gaussian fitted to each scenario (+: mean). PCA fitted on the standardized intact data.

Se verifica que el coeficiente de correlación entre PC1 y PC2 en el escenario intacto sea cero. Esto es lo esperado, porque el PCA se ajustó con los datos de ese escenario y sus componentes, por construcción, tienen covarianza nula entre sí, es decir, no están correlacionados.

```
# Correlation coefficient between PC1 and PC2 for the intact data (must be 0:
fundamental PCA property)
rho_pc12 = np.corrcoef(pc12['intact'][:, 0], pc12['intact'][:, 1])[0, 1]
print(f'Correlation coefficient between PC1 and PC2 (intact): rho = {rho_pc12:.2e}')
```

```
Correlation coefficient between PC1 and PC2 (intact): rho = 3.67e-17
```

```
# 2D Wasserstein distance between the PC1-PC2 scores of the intact scenario and each
damage scenario
w_pc12 = {dmg: stats.wasserstein_distance_nd(pc12['intact'], pc12[dmg]) for dmg in
DAMAGES}

w_pc12_table = pd.DataFrame({f'W (intact, {dmg})': [w_pc12[dmg]] for dmg in DAMAGES},
                            index=pd.Index(['PC1-PC2'], name='PCA space'))
show_table(w_pc12_table, 17, 'Two-dimensional Wasserstein distance between the PC1-PC2
scores of the intact scenario '
           'and each damage scenario.', 'E3_wasserstein_PC1_PC2.csv', precision=3)
```

Table 17. Two-dimensional Wasserstein distance between the PC1-PC2 scores of the intact

scenario and each damage scenario.

PCA space	W (intact, damage1)	W (intact, damage2)
PC1-PC2	4.175	12.809

Al agregar la componente PC2 se aumenta siginificativamente la sensibilidad a la detección (de 0.479 a 4.175 en el daño 1 y de 1.970 a 12.809 en el daño 2). Esto sugiere que la PC1 conserva principalmente la información dada por las frecuencias naturales y la PC2 la información dada por la variación de los MACs. Pese a que la distancia de Wasserstein aumenta a medida que se agregan variables o dimensiones, el aumento observado al agregar la PC2 no se explica solamente con el aumento de las dimensiones.

E4. Progressive Dimensionality Impact (3 pts)

```
# Scores of every scenario on all 6 PCs (PCA fitted on the intact data)
# pc_all[scenario]: (N_MEAS, 6) array -> measurements x [PC1, ..., PC6]
pc_all = {sc: pca.transform(features_std[sc]) for sc in SCENARIOS}

# Wasserstein distance between intact and each damage scenario using the first k PCs,
k = 1, ..., 6
K = np.arange(1, len(FEATURES) + 1)
w_k = {dmg: np.array([stats.wasserstein_distance_nd(pc_all['intact'][:, :k],
pc_all[dmg][:, :k]) for k in K])
       for dmg in DAMAGES}

w_k_table = pd.DataFrame({f'W (intact, {dmg})': w_k[dmg] for dmg in DAMAGES},
                         index=pd.Index(K, name='Number of PCs k'))
show_table(w_k_table, 18, 'Wasserstein distance between the intact scenario and each
damage scenario using the first '
           '$k$ principal components, $k$ = 1, ..., 6.', 'E4_wasserstein_k_PCs.csv',
precision=3)
```

Table 18. Wasserstein distance between the intact scenario and each damage scenario using the first k principal components, k = 1, ..., 6.

Number of PCs k	W (intact, damage1)	W (intact, damage2)
1	0.479	1.970
2	4.175	12.809
3	4.707	14.601
4	4.750	14.733
5	4.758	14.738
6	4.764	14.742

Con los resultados de la tabla 18, se puede decir que la sensibilidad a la detección se empieza a aplanar a partir de k= 3, justo donde las componentes del PCA representan más del 95% de la varianza de los datos.

Para comparar el resultado de las 6 componentes del PCA con las 6 características sin PCA, se calcula la distancia de Wasserstein de las 6 características estandarizadas para ambos escenarios de daño.

```
# 6D Wasserstein distance with all standardized features (no PCA), to compare with k = 6 PCs
w_6d = {dmg: stats.wasserstein_distance_nd(features_std['intact'], features_std[dmg])
for dmg in DAMAGES}
for dmg in DAMAGES:
    print(f'{dmg}: 6 features (no PCA) = {w_6d[dmg]:.3f}, 6 PCs = {w_k[dmg][-1]:.3f}')
```

```
damage1: 6 features (no PCA) = 4.764, 6 PCs = 4.764
damage2: 6 features (no PCA) = 14.742, 6 PCs = 14.742
```

Como se puede observar, los resultados son idénticos. Esto se debe a que las 6 componentes del PCA conservan el 100 % de la varianza de los datos originales, solo que en otros ejes.

E5. Physical Interpretation (2 pts)

Los pesos (loadings) de la PC1 se obtienen de la descomposición en valores y vectores propios de la matriz de covarianza de las 6 características estandarizadas del escenario intacto:

$$\mathbf{\Sigma}_0 = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \tag{4}$$

Donde:

- $\mathbf{\Sigma}_0$: matriz de covarianza (6 × 6) de las características estandarizadas del escenario intacto.
- $\mathbf{V}$: matriz cuyas columnas son los vectores propios; cada columna define la dirección de una componente principal.
- $\mathbf{\Lambda}$: matriz diagonal con los valores propios, es decir, la varianza de cada componente, ordenadas de mayor a menor.

Los loadings de la PC1 son los 6 valores del primer vector propio, $\mathbf{v}_1 = [v_{11}, \ldots, v_{16}]$. En el código, estos valores corresponden a `pca.components_[0]`.

```
# PC1 loadings: weight of each standardized feature in PC1 (first eigenvector of the intact covariance)
pc1_loadings = pd.DataFrame({'PC1 loading': pca.components_[0]},
index=pd.Index(FEATURES, name='Feature'))
show_table(pc1_loadings, 19, 'Loadings of the first principal component (PCA fitted on the 6 standardized features of '
           'the intact scenario).', 'E5_PC1_loadings.csv', precision=3)
```

Table 19. Loadings of the first principal component (PCA fitted on the 6 standardized features of the intact scenario).

Feature	PC1 loading
f1	0.542
f2	0.542

Feature	PC1 loading
f3	0.544
MAC1	0.163
MAC2	-0.235
MAC3	-0.186

En la tabla 19 se observa que las features que mayor aportan al PC1 son las frecuencias naturales. Esto explica tambien los resultados obtenidos en la parte E1 donde las distancias de Wasserstein calculadas para el PC1 fueron bajas.

Considerando que la variación de las frecuencias naturales vienen dadas principalmente por variación global en la rigidez o masa de las estructuras, se puede decir que el PC1 representa fisicamente la relación masa-rigidez general de la estructura.

De acuerdo a los resultados obtenidos en la sección E4, la reducción de la dimensionalidad de los datos funciona para este tipo de problemas y se puede concluir que la varianza descartada por las últimas componentes corresponden principalmente a la varianza dada por errores en la estimacion de las características o ruido.

Part F: Engineering Insights (5 pts)

F1. Method Comparison and Selection (2 pts)

F2. Feature Selection Strategy (2 pts)

F3. Practical Implementation Considerations (1 pt)

Use of Generative AI